

Swapping Data

Swapping columns in a NumPy array using indexing is a technique where you change the order of columns

Example

```
import numpy as np
```

```
arr = np.array([1,2,3,4,5])  
arr[0], arr[2] = arr[2], arr[0]  
print(arr)
```

Swapping Columns Using Indexing

Swapping within a 2D array

Example (Row swapping)

```
import numpy as np  
  
arr = np.array([[1,2,3,4],[5,6,7,8],[9,10,11,12]])  
print(arr)  
arr[[0,2],:] = arr[[2,0],:]  
print(arr)
```

Example (column swapping)

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

print(arr)

arr[:, [0, 2]] = arr[:, [2, 0]]

print(arr)
```

Output

```
[[3 2 1]
```

```
[6 5 4]
```

```
[9 8 7]]
```

Swapping Multiple Columns

Swapping multiple columns in a NumPy array involves changing the order of more than two columns simultaneously. The process uses advanced indexing to specify which columns to swap and their new positions.

Example

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Swap columns in the order: 1st to 3rd, 2nd to 1st, 3rd to 2nd

arr[:, [0, 1, 2]] = arr[:, [1, 2, 0]]

print(arr)
```

Output

```
[[2 3 1]
 [5 6 4]
 [8 9 7]]
```

Swapping Columns in a 3D Array

Swapping columns in a 3D array in NumPy involves rearranging the elements along one of the axes of the array, usually the second axis (axis 1), which corresponds to the columns when dealing with a slice of the 3D array.

Example

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]],
                [[7, 8, 9], [10, 11, 12]]])
```

```
arr[:, :, [0, 2]] = arr[:, :, [2, 0]]

print(arr)
```

Output

```
[[[ 3  2  1]
  [ 6  5  4]
  [ 9  8  7]
  [12 11 10]]]
```

Handling Missing Data in Arrays

Handling missing data is a common challenge in data analysis and processing. Missing data in arrays can arise due to various reasons, such as incomplete data collection, errors during data entry, or intentional omission.

In NumPy and data analysis, dealing with missing values involves identifying, handling, and processing them effectively to ensure data integrity and accurate results.

Identifying Missing Data

To handle missing data, the very first step is to identify it. In NumPy, missing values are often represented as `np.nan` in floating-point arrays. You can use specific functions such as `np.isnan()` to detect these missing values.

Example

```
import numpy as np

arr = np.array([1, 2, np.nan, 4, np.nan, 6])

# Checking for missing values

is_nan = np.isnan(arr)

print(arr)

print(is_nan)
```

Output

```
[ 1.  2. nan  4. nan  6.]
```

```
[False False  True False  True False]
```

Removing Missing Data

Removing missing data involves eliminating parts of your dataset where data is missing.

In NumPy, you can use boolean indexing to exclude NaN values from arrays. For example, creating a mask that identifies missing values and then using it to filter out those values.

Example

```
import numpy as np
```

```
arr = np.array([1, 2, np.nan, 4, np.nan, 6])
```

```
arr2 = arr[~np.isnan(arr)]
```

```
print(arr)
```

```
print(arr2)
```

Output

```
[ 1.  2. nan  4. nan  6.]
```

```
[1.  2.  4.  6.]
```

Replacing Missing Data

Replacing missing data means filling in the gaps where data is missing with a substitute value. In NumPy, you can use the `np.nan_to_num()` function to replace NaN values with a specific number, such as zero or the mean of the other values. Following is the syntax –

```
numpy.nan_to_num(x, copy=True, nan=0.0, posinf=None, neginf=None)
```

Where,

x: The input array containing NaN values, infinities, or other numerical values.

copy: A boolean indicating whether to make a copy of the array (True by default). If False, the operation may be performed in place.

nan: The value to replace NaN values with. The default is 0.0.

posinf: The value to replace positive infinity (inf) with. If not specified, it defaults to a very large number.

neginf: The value to replace negative infinity (-inf) with. If not specified, it defaults to a very small (negative) number.

Example

```
import numpy as np
```

```
arr = np.array([1, 2, np.nan, 4, np.nan, 6])
```

```
arr2 = np.nan_to_num(arr, nan=0)
```

```
print(arr)
```

```
print(arr2)
```

Output

```
[ 1.  2. nan  4. nan  6.]
```

```
[1.  2.  0.  4.  0.  6.]
```

I/O with NumPy

I/O in NumPy refers to input/output operations, allowing you to save and load arrays to and from files. Functions like `np.save()` and `np.load()` handle binary files, while `np.savetxt()` and `np.loadtxt()` work with text files.

File Formats in NumPy

NumPy provides support for saving and loading arrays in various file formats extensively. These formats allow you to store data in a manner that is easy to share, read, and process.

When working with large datasets, it is important to choose the appropriate file format for storing your NumPy arrays. The most common formats are:

.npy	The .npy format is the native binary format used by NumPy to store arrays. This format preserves the array's data type and shape information.
-------------	---

numpy.save() Function

<code>numpy.save()</code>	function saves a single NumPy array in the .npy format. This format is highly efficient for storing large arrays as it keeps all metadata intact.
---------------------------	---

Example

This example, we will create a NumPy array and save it to a .npz file –

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

np.save('data.npy', arr)

array_data = np.load('data.npy')

print(array_data)
```

Output

```
[[1 2 3]
 [4 5 6]]
```

.npz

The .npz format is a compressed archive used to store multiple arrays in one file. It is helpful when working with several arrays, as it allows you to save them together in a single, compressed file.

numpy.savez() Function

`numpy.savez` function saves multiple arrays in a single `.npz` file. You can also use `numpy.savez_compressed()` function to compress the data within the archive.

Example

Save two arrays into a single `.npz` file and load them back –

```
import numpy as np

arr1 = np.array([1, 2, 3])

arr2 = np.array([[4, 5, 6], [7, 8, 9]])

np.savez('data.npz', array1=arr1, array2=arr2)

data = np.load('data.npz')

print("array1: ", data['array1'])

print("array2: ", data['array2'])
```

Output

```
array1: [1 2 3]
```

```
array2: [[4 5 6]
```

```
[7 8 9]]
```

Text Files For human-readable data storage, including CSV and custom text formats.

Reading and Writing Text Files

Text files are one of the most common ways to store data. NumPy provides functions to read from and write to text files. The primary functions for these operations are `numpy.loadtxt()` and `numpy.savetxt()`.

The `numpy.loadtxt()` Function

The `numpy.loadtxt()` function loads data from a text file into a NumPy array. It can handle different data types and allows customization of delimiters and skipping of header lines.

Example

Loading data from a text file in NumPy using the `np.loadtxt()` function –

```
import numpy as np
```

```
# Create a sample text file

with open('data.txt', 'w') as f:

    f.write("1.0 2.0 3.0\n4.0 5.0 6.0\n7.0 8.0 9.0")

# Load the data from the text file

data = np.loadtxt('data.txt')

print(data)
```

Output

```
[[1. 2. 3.]
 [4. 5. 6.]
 [7. 8. 9.]]
```

The numpy.savetxt() Function

The `numpy.savetxt()` function saves a NumPy array to a text file. It allows customization of the delimiter, format, and header information for the output file.

Example

Saving a NumPy array to a text file in using the `np.savetxt()` function –

```
import numpy as np

data = np.array([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0], [7.0, 8.0, 9.0]])

# Save the array to a text file

np.savetxt('output.txt', data, delimiter=' ', fmt='%1.1f')
```

The data is saved to the file 'output.txt' with space as the delimiter and one decimal point format –

Handling CSV Files

Comma-Separated Values (CSV) files are widely used for data storage and exchange. NumPy can read and write CSV files using `numpy.genfromtxt()` and `numpy.savetxt()` functions.

numpy.genfromtxt() Function

The `numpy.genfromtxt()` function loads data from a text file, handling missing values and non-numeric data. It is more flexible than `loadtxt()` function, allowing for advanced customization like filling missing values and specifying data types.

Example

Creating a CSV file with sample data and saving it. Then, we use `np.genfromtxt()` to load the data from the CSV file into a NumPy array and print it –

```
import numpy as np

with open('data.csv', 'w') as f:

    f.write("1.0,2.0,3.0\n4.0,5.0,6.0\n7.0,8.0,9.0")


# Load the data from the CSV file

data = np.genfromtxt('data.csv', delimiter=',')

print(data)
```

Output

```
[[1. 2. 3.]
```

```
[4. 5. 6.]
```

```
[7. 8. 9.]]
```


Solving Linear Equations in NumPy

NumPy provides several methods to solve linear equations. The most commonly used method is by using the `numpy.linalg.solve()` function, which directly solves the system of linear equations.

Example

In the following example, the system of equations –

$$3x + 2y = 5$$

$$x + 2y = 5$$

has been solved to give the solution $x = 0$ and $y = 2.5$

```
import numpy as np
```

```
arr1 = np.array([[3, 2], [1, 2]])
```

```
arr2 = np.array([5, 5])
```

```
res = np.linalg.solve(arr1, arr2)
```

```
print(res)
```

Output

```
[0. 2.5]
```

The `numpy.linalg.solve()` Function

The `numpy.linalg.solve(A, b)` function computes the solution to the linear system $\text{arr1} * x = \text{arr2}$. The function takes two arguments:

`arr1`: The coefficient matrix (2D NumPy array).

`arr2`: The constant vector (1D NumPy array).

The function returns the solution vector `x` that satisfies the equation. The function uses efficient methods, such as Gaussian elimination or LU decomposition, to solve the system.

Alternative Ways to Solve Linear Equations

In addition to `numpy.linalg.solve()` function, NumPy provides other ways to solve linear equations, such as using matrix inversion or the `numpy.dot()` function.

These methods are useful when you need more control over the solving process or want to explore the mathematical background of solving linear systems.

Using Matrix Inversion

One way to solve a system of linear equations is by finding the inverse of the coefficient matrix A . If A is invertible, the solution can be found by multiplying the inverse of A with the vector b :

```
arr1 = arr-1 * arr2
```

NumPy provides the `numpy.linalg.inv()` function to compute the inverse of a matrix. Let us see how to use it –

```
import numpy as np
```

```
arr1 = np.array([[3, 2], [1, 2]])
```

```
arr2 = np.array([5, 5])
```

```
# Compute the inverse of A
```

```
arr1_inv = np.linalg.inv(arr1)
```

```
# Solve the system of equations using matrix inversion
```

```
res = np.dot(arr1_inv, arr2)
```

```
print(res)
```

Output

```
[4.4408921e-16 2.5000000e+00]
```

Graph Plotting

Matplotlib is a Python library used to create high-quality plots and charts. It is highly customizable and can produce various types of plots, such as line plots, scatter plots, bar plots, and histograms.

Setting Up Matplotlib

```
pip install matplotlib
```

or

```
python -m pip install matplotlib
```

Line Plot

A line plot is one of the simplest and most commonly used visualizations. It is used to show trends or relationships between data points.

Example

```
import numpy as np

import matplotlib.pyplot as plt

# 100 evenly spaced points between 0 and 10
x = np.linspace(0, 10, 100)

# Compute sine values for x
y = np.sin(x)

# Create a line plot
plt.plot(x, y, label='sin(x)', color='red', linestyle='--')

plt.title('Line Plot of sin(x)')

plt.xlabel('X-axis')

plt.ylabel('Y-axis')

plt.legend()

plt.grid(True)

plt.show()
```

Scatter Plot

Scatter plots are used to display relationships between two variables by showing individual data points. They are useful for identifying patterns, correlations, or outliers in data.

Example

```
import numpy as np

import matplotlib.pyplot as plt

# Generate random data

x = np.random.rand(50)

y = np.random.rand(50)

# Create a scatter plot

plt.scatter(x, y, color='red', alpha=0.7)

plt.title('Scatter Plot of Random Points')

plt.xlabel('X-axis')

plt.ylabel('Y-axis')

plt.show()
```

Bar Plot

Bar plots are used to compare different categories or groups. They display data using rectangular bars, where the height represents the value.

Example

```
import numpy as np

import matplotlib.pyplot as plt

# Data for bar plot
categories = ['A', 'B', 'C', 'D']
values = [10, 20, 15, 25]

# Create a bar plot
plt.bar(categories, values, color='orange')

plt.title('Bar Plot Example')

plt.xlabel('Categories')

plt.ylabel('Values')
```



```
plt.show()
```

Histogram

Histograms are used to visualize the frequency distribution of a dataset. They divide data into intervals (bins) and show the count of data points in each interval.

Example

```
import numpy as np

import matplotlib.pyplot as plt

# Generate random data

# 1000 samples from a normal distribution

data = np.random.randn(1000)

# Create a histogram

plt.hist(data, bins=30, color='purple', alpha=0.8)

plt.title('Histogram of Random Data')
```

```
plt.xlabel('Bins')
```

```
plt.ylabel('Frequency')
```

```
plt.show()
```

Pie Chart

Pie charts are used to represent data as slices of a circle, showing proportions or percentages. While not always the best for precise comparisons, they can be visually appealing for specific use cases.

Example

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Data for pie chart
```

```
labels = ['Python', 'Java', 'C++', 'Ruby']
```

```
sizes = [50, 30, 15, 5]
```

```
# Create a pie chart
```

```
plt.pie(sizes, labels=labels, autopct='%1.1f%%', startangle=140)
```

```
plt.title('Pie Chart ')
```

```
plt.show()
```

Customizing Matplotlib Visualizations

Matplotlib offers extensive options to customize your visualizations. You can adjust colors, line styles, markers, fonts, and more.

Example

Following is an example of creating a customized plot with different line styles, colors, and markers for sine and cosine waves –

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# Customized line plot
```

```
x = np.linspace(0, 10, 100)
```

```
y1 = np.sin(x)
```

```
y2 = np.cos(x)
```

```
plt.plot(x, y1, label='sin(x)', color='red', linewidth=2, marker='o')
```

```
plt.plot(x, y2, label='cos(x)', color='blue', linestyle='dotted')
```

```
plt.title('Customized Line Plot')
```

```
plt.xlabel('X-axis')
```

```
plt.ylabel('Y-axis')
```

```
plt.legend()
```

```
plt.grid(True, linestyle='--', color='gray', alpha=0.6)
```

```
plt.show()
```

Combining NumPy and Matplotlib

Using NumPy and Matplotlib together can enhance your analysis and visualization workflow. NumPy can be used to preprocess and manipulate data, while Matplotlib can be used to visualize the results.

For instance, you can generate data transformations or statistical analyses using NumPy and then visualize them with Matplotlib.

Example

```
import numpy as np
```

```
import matplotlib.pyplot as plt

# Generate and visualize a quadratic function

x = np.linspace(-10, 10, 200)

y = x**2 - 5*x + 6

plt.plot(x, y, label='y = x^2 - 5x + 6', color='magenta')

plt.title('Quadratic Function Visualization')

plt.xlabel('X-axis')

plt.ylabel('Y-axis')

plt.legend()

plt.show()
```